

Question 1

Recall that the factorial function $n!$ returns $n! = n \times (n-1) \times \dots \times 2 \times 1$. Write a function that executes this called *factorial2* that uses a for loop.

Question 2

Compute an approximation for π using Monte Carlo. You may only use `rand()` for random number generation. **Hint:** If U is a bivariate uniform random variable on the unit square $(0, 1)^2$, then the probability that U lies in a subset B of $(0, 1)^2$ is equal to the area of B .

Question 3

Consider the correlated time series

$$x_{t+1} = \alpha x_t + \varepsilon_{t+1},$$

where $x_0 = 0$ and $t = 0, \dots, n$. The sequence of shocks ε_t is assumed to be standard normal iid.

- Let $n = 200$ and simulated and plot time series for $\alpha = 0$, $\alpha = 0.8$, $\alpha = 0.9$, and $\alpha = 0.98$.
- Comment on the differences in unconditional volatilities across these simulations.

Question 4

Let the data generating process for y be

$$y = ax_1 + bx_1^2 + cx_2 + d + \sigma w,$$

where y, x_1, x_2 are scalars, a, b, c, d are parameters to estimate, and $w \sim N(0, 1)$ iid.

- First, draw $N = 50$ values for x_1, x_2 from iid normal distributions.
- Then, draw a w vector for the N values to generate y for the simulated data. Use $a = 0.1$, $b = 0.2$, $c = 0.5$, $d = 1.0$, and $\sigma = 0.1$. Repeat draws of w until you have 200 different simulations of the y values.
- Finally, calculate ordinary least squares manually for each of the 200 simulations. Plot histograms for each parameter a, b, c, d .

Question 5

The root of a univariate function $f(\cdot)$ is an x such that $f(x) = 0$. One solution method for finding such roots is known as Newton's method: starting with an x_0 guess, a function $f(\cdot)$ and its derivative $f'(\cdot)$, the algorithm is to repeat:

$$x^{n+1} = x^n - \frac{f(x^n)}{f'(x^n)}$$

until $|x^{n+1} - x^n|$ is below a tolerance.

- Write a function to implement Newton's method, where the function would accept arguments f , f_{prime} , x_0 , tolerance, and max iterations.
- Test the function with $f(x) = (x - 1)^3$ and another function of your choice where you can analytically find the derivative.

Question 6

Debug the code in “debug-exercise-pipeline.jl” using print statements.

Question 7

The following pseudocode implements merge sort but contains a bug. Implement this code in Julia, then use a debugger to find and fix the bug.

```
function merge_sort(arr):
    if length(arr) <= 1:
        return arr
    mid = length(arr) / 2
    left = merge_sort(arr[0..mid])
    right = merge_sort(arr[mid..end])
    return merge(left, right)

function merge(left, right):
    result = []
    i = 0, j = 0
    while i < length(left) AND j < length(right):
        if left[i] <= right[j]:
            append result, left[i]
            i = i + 1
        else:
            append result, right[i]
            j = j + 1
    append remaining elements from left and right
    return result
```

Test vector: “merge_sort([3, 1, 4, 1, 5, 9, 2, 6])” should return “[1, 1, 2, 3, 4, 5, 6, 9].” Use breakpoints and step through the merge function to find where the incorrect element is being selected.

Question 8

Debug the code in “debug_exercise.jl.” Use an AI of your choice to interpret some of the error messages in the earlier examples and to explain the mistakes made in some of the later examples. How well does it do?

Question 9

This question will take you through using Git, Github, and Github Desktop collaboratively to improve the performance of some Julia code. The “perf_exercise.jl” file on Google Drive contains several Julia functions that have been deliberately unoptimized — that is, made to be slower than they could be.

- Form a pair with a partner. Open a Github repository with “perf_exercise.jl” in it, and then fork your partner’s repository.
- Clone your partner’s repository to your computer. Edit their code to be as fast as possible.
- Commit your changes to their code and then send them a pull request to incorporate your changes.

Question 10

“Vibe code” some of the exercises on the first page of this problem set by describing the exercises to an AI without actually writing any of the code yourself. Experiment with how explicit you need to make your directions. How does it go?